Lecture 08: IP Addressing and DNS

Objectives:

- Learn how IP address is used to Identify network nodes
- Learn how DNS is used to map domain-names to IP addresses
- Learn how to find IP address of a node using C#

1. IP Addresses

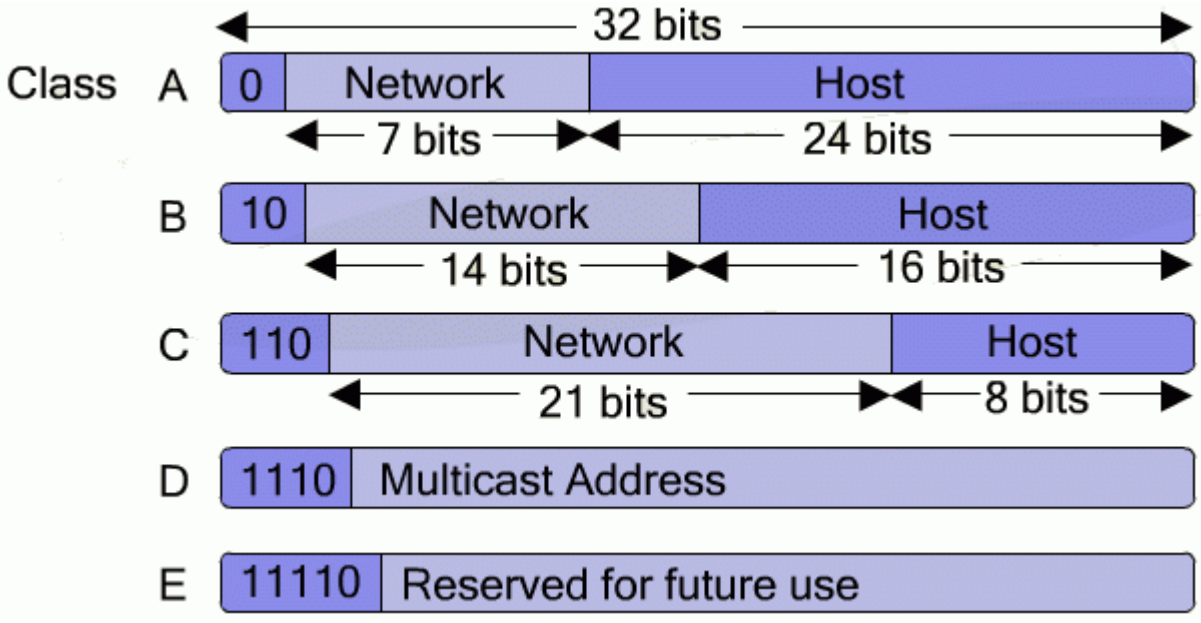
Every node (***host*** or ***router*****)** on the Internet has a unique IP address, which is used to identify it.

IP addresses are 32 bits in length. At the moment, an address length of 32 bits provides adequate coverage. However, this is unlikely to be true in the future.

Accordingly, plans are underway to extend the 32 bit address range to cater for a much larger number of nodes (IPv6 – 128bits).

In order to ensure that no two machines mistakenly use the same IP, the allocation of IP addresses is done by the International Network Information Center (***Internic***).

Presently, IP addresses are separated into five different classes (although only four classes are actually used). As follows:



IP addresses mostly consist of two different parts: a **Network-dentifier**

and a **Host-identifier** -the Network Identifier identifies the network to which the IP belongs, and the Host Identifier identifies the particular host within that network.

IPs are normally written in **dotted decimal format** (8 bits at a time, separated by a dot).

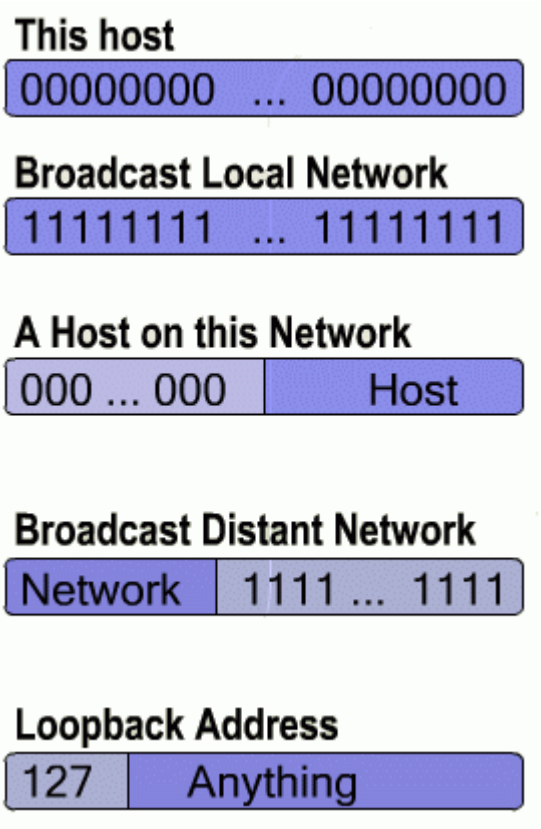
```
11000100 00000001 01000001 10001111 = 196.1.65.143
```

The class ranges are as follows:

- **Class A** provides support for 126 networks, each with up to 16 million hosts (IP range: 1.0.0.0 to 127.255.255.255).
- **Class B** provides support for 16,382 networks, each with up to 64k hosts (IP range: 128.0.0.0 to 191.255.255.255).
- **Class C** provides support for 2 million networks, each with up to 254 hosts (IP range: 192.0.0.0 to 223.255.255.255).
- **Class D** IP addresses are used for multicasting, i.e. sending IP packets to multiple hosts (IP range: 224.0.0.0 to 239.255.255.255).
- **Class E** IP addresses are reserved for future use (IP range: 240.0.0.0 to 247.255.255.255).

A number of special IP addresses are defined:

- **0.0.0.0** refers to the current host.
- **255.255.255.255** denotes a broadcast to all machines on the local network.
- A *network* number of **0** enables hosts to refer to their own network without knowing the exact network number.
- A *host* address consisting of all 1's denotes a broadcast upon a distant network as specified by the network number.
- An IP address starting with **127.xxx.yyy.zzz** is considered as a ***loopback*** address (this is primarily for debugging network applications).



2. Overview of Domain Name Services (DNS)

DNS provides a means of mapping between a numeric IP address (which is hard to remember) and an ASCII string (which is easier to remember).

For example, would you rather try to remember ***www.google.com*** or ***216.239.39.100?***

DNS is structured in such a way that a network consisting of millions of hosts can be efficiently managed.

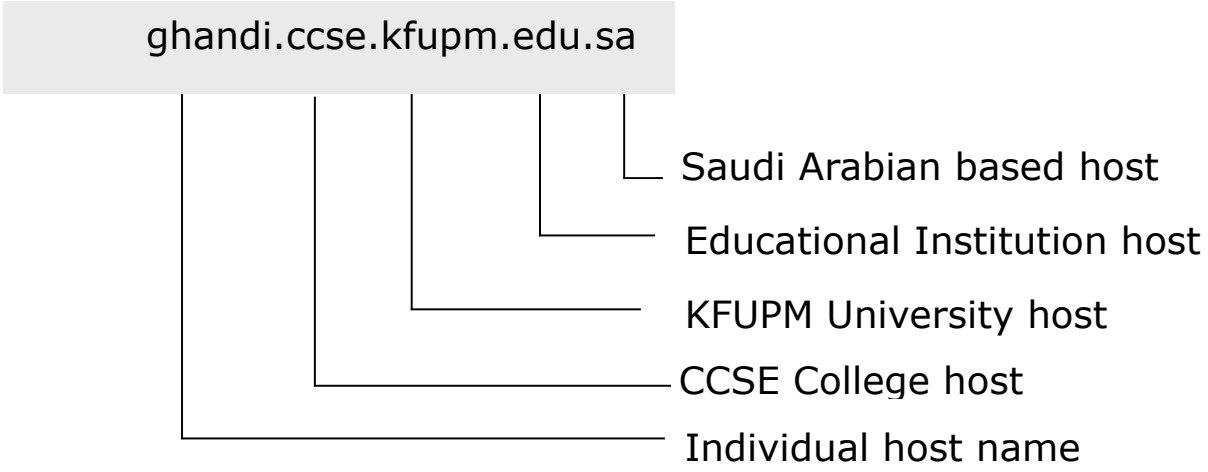
An application that wishes to resolve a DNS name onto an IP address, sends the hostname to a ***resolver***. The resolver sends a UDP packet to a **local DNS server**, which then looks up the name and returns the IP address to the resolver, which in turn passes it on to the application.

The application can then use the IP address to establish a TCP connection or send UDP packets, etc.

DNS name space organization

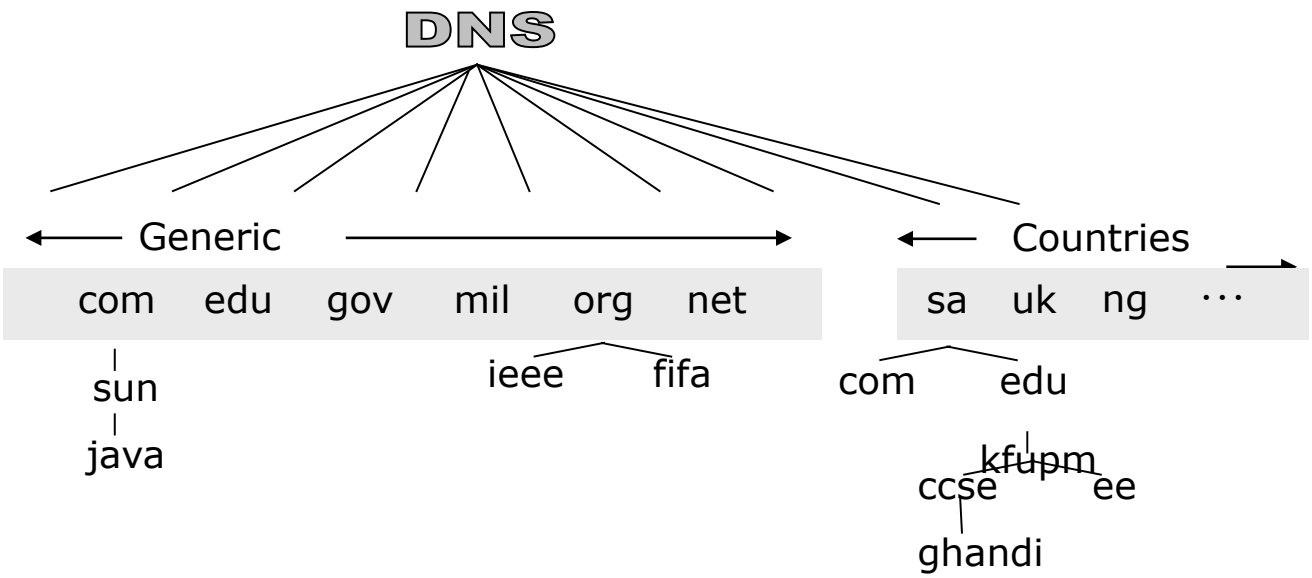
In order to cater for millions of hosts, DNS uses a hierarchical, domain based naming scheme with a distributed database system used to implement the naming scheme.

In other words, DNS is like a worldwide postal service. Consider, for example, the following DNS address:



Each part of the address narrows down the range of possible hosts, until an individual host is found.

The Internet is divided into several ***top-level domains***, with each domain covering a number of related hosts. Each domain is then further subdivided (in a tree like structure) until individual hosts are reached, e.g.



Two different types of top level domain are defined: namely generic names and country names.

A specific country domain is provided for every single country (as defined in ISO 3166). Generic top-level domains are as follows:

Generic DNS Names			
com	Commerical sites	mil	US armed forces
gov	US government	net	Network providers
edu	Educational institutions		
int	International organisations		
org	Non-profit organisations		

Note that domain names are case insensitive, i.e. **WWW.SUN.COM** is the same as **www.sun.com.** Additionally, component names can be up to 63

characters in length, with the full path name not longer than 255 characters.

Name Servers

In principle, a single name server could be used to store the entire DNS database and respond to any name lookup requests.

However, in practice, this server would be too overloaded to be usable, and should the server go down, then the entire Internet would be crippled.

To overcome these problems, the DNS name space is divided up into a number of **non-overlapping** regions called *zones*.

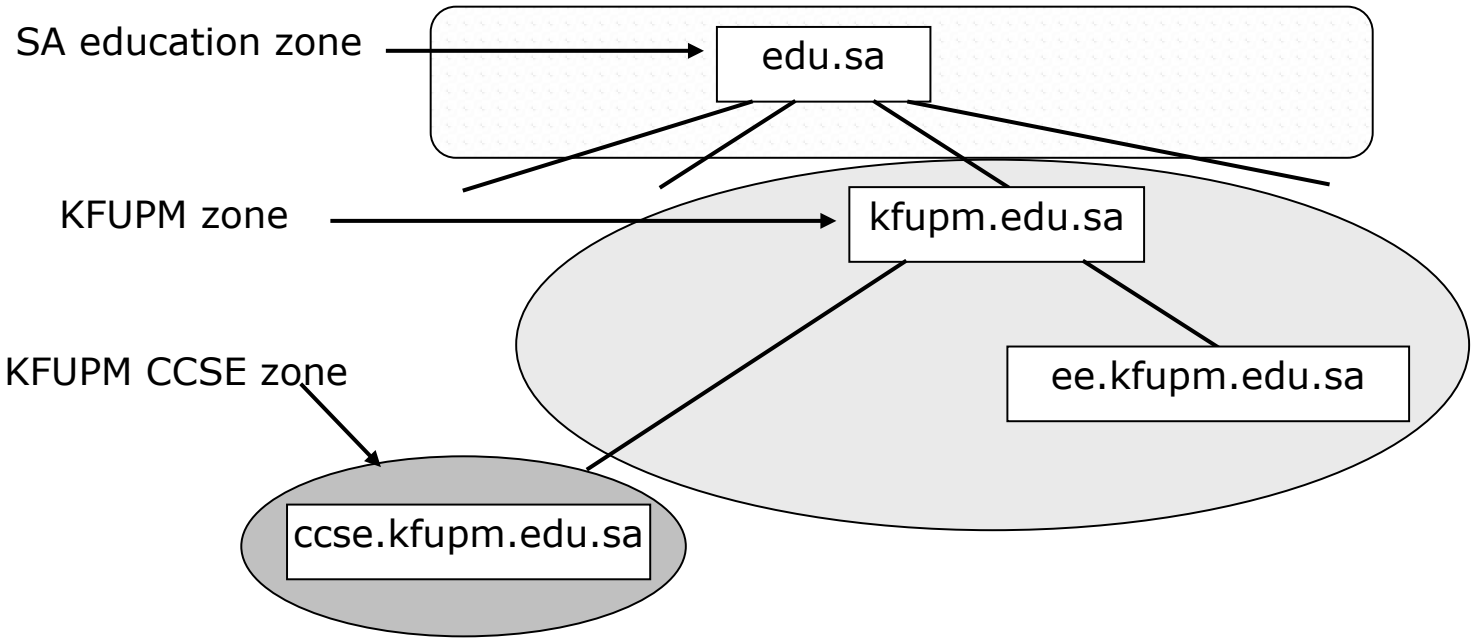
Each zone contains part of the entire DNS namespace tree, along with a number of domain name servers which manage the zone.

Normally a zone will have one primary name server, which is responsible for maintaining details of all the hosts within the zone.

In addition to the primary server, a number of secondary servers which mirror the information contained on the primary server are used.

For example, a university will encompass one zone (e.g. kfupm.edu.sa), having a central server.

Within the university, the college of computer science & engeneering might wish to administer its own zone, i.e. ccse.kfupm.edu.sa is a separate domain with its own server. Whilst, the ee department may not want to maintain a separate zone, i.e. ee.kfupm.edu.sa is part of the kfupm.edu.sa zone and managed by the kfupm.edu.sa central server.



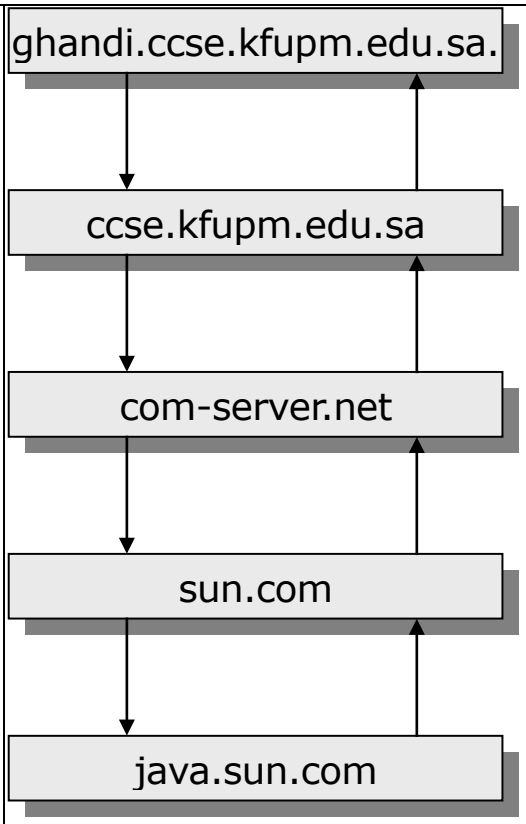
How DNS lookup works

When a ***resolver*** (a daemon which handles DNS lookup on behalf of an application) has a query, it passes it onto a local name server.

If the requested hostname falls within the bounds of the local zone, then an **authoritative resource record** is returned.

If the specified hostname is not known to the local name server, then the query is passed to the top level name server of the requested domain, e.g.

- A resolver running on 'ghandi.' wishes to determine the IP address of 'download.java.sun.com'. It firstly sends a query to the local name server ccse.kfupm.edu.sa.
- Assuming the local DNS server has no knowledge of the requested host, it will then forward the query to the top-level *com* server.
- The top level *com* server is unlikely to exactly know the requested IP. Hence, the request is forwarded to *sun.com*. In turn, the request will arrive at *java.sun.com* which will contain the authoritative resource record for the requested host.
- Once found, the reply works its way back up through the chain until it reaches ccse.kfupm.edu.sa, which passes on the information to the resolver running on ghandi.ccse.kfupm.edu.sa.
- Also note, most DNS servers will maintain a cache of recently requested IPs, i.e. ccse.kfupm.edu.sa will retain the IP of *download.java.sun.com* in case it is requested again. However, the cached addresses will not be authoritative, hence a **Time_to_live** field is defined, specifying how long the record should remain within the cache.



3. Finding IP Address

When sending data across the network, you often need to determine the IP network information for the system running your program.

The Windows OS family offers many ways to determine IP configuration information, both manually and from within a program. We look at few of these methods:

A. Using ipconfig

ipconfig is a command line programs that comes with the OS. It can be used to display IP network information for each active network interface on the current system. It has many options, but the most commonly used are:

Z: \>ipconfig	Displays basic IP network information about the host on which it is run
Z: \>ipconfig /all	Displays detailed IP network information about the host on which it is run

```
C:\Documents and Settings\ics>ipconfig /all

Windows IP Configuration

        Host Name . . . . . . . . . . . . : ics-bmghandi
        Primary Dns Suffix  . . . . . . . :
        Node Type . . . . . . . . . . . . : Hybrid
        IP Routing Enabled. . . . . . . . : No
        WINS Proxy Enabled. . . . . . . . : No
        DNS Suffix Search List. . . . . . : pc.ccse.kfupm.edu.sa
                                            ccse.kfupm.edu.sa
                                            kfupm.edu.sa

Ethernet adapter Local Area Connection:

        Connection-specific DNS Suffix  . : pc.ccse.kfupm.edu.sa
        Description . . . . . . . . . . . : Intel(R) PRO/100 VE Network Connecti
on
        Physical Address. . . . . . . . . : 00-08-02-BC-4A-30
        Dhcp Enabled. . . . . . . . . . . : Yes
        Autoconfiguration Enabled . . . . : Yes
        IP Address. . . . . . . . . . . . : 196.1.65.143
        Subnet Mask . . . . . . . . . . . : 255.255.255.0
        Default Gateway . . . . . . . . . : 196.1.65.253
                                            196.1.65.254
        DHCP Server . . . . . . . . . . . : 196.1.65.21
        DNS Servers . . . . . . . . . . . : 196.1.65.21
                                            172.16.0.1
                                            196.1.64.3
        Primary WINS Server . . . . . . . : 196.1.65.21
        Secondary WINS Server . . . . . . : 172.16.0.1
        Lease Obtained. . . . . . . . . . : Sunday, October 05, 2003 8:39:57 PM
        Lease Expires . . . . . . . . . . : Monday, October 06, 2003 1:39:57 AM
```

B. Using nslookup

nslookup is a command line programs that also comes with the OS. It can be used to query the local DNS server about any host on the internet.

Example:

```
C:\Documents and Settings\ics>nslookup www.microsoft.com
Server:  ccse-tungsten.pc.ccse.kfupm.edu.sa
Address:  196.1.65.21

Non-authoritative answer:
Name:    a562.cd.akamai.net
Addresses:  81.52.248.159, 81.52.248.161, 81.52.248.166, 81.52.248.142
          81.52.248.145, 81.52.248.151, 81.52.248.152, 81.52.248.153, 81.52.248.
158
Aliases:  www.microsoft.com, www.microsoft.com.edgesuite.net
```

We note the following points from the above results:

1. The first two lines displays info about the local DNS sever reporting the query result.
2. *Non-authoritative* indicates that this information is obtained from the cache of the local DNS server.
3. Name indicates the actual name of the machine – thus, www.microsoft.com is just an alias as we can see from the last line.
4. All IPs associated with the machine are returned.

C. Using DNS class in C#

C# provides the **DNS** class in the **System.Net** namespace, which provides static methods that can be used to query the local DNS server to resolve IPs and domain names.

Some of the methods of the DNS class are:

`static string GetHostName()`
`static IPHostEntry GetHostByName(string hostname)`
`static IPHostEntry GetHostByAddress(IPAddress address)`
`static IPHostEntry GetHostByAddress(string address)`
`static IPHostEntry Resolve(string hostName)`

GetHostName returns the name of the current host.

The rest of the methods return an instance of **IPHostEnty** class. This class has three properties which are used to retrieve the information about the host, Namely:

AddressList	Returns an array of IPAddress objects, one for each IP Address of the machine.
Aliases	Returns An array of string objects, one for each alias.
HostName	Returns a string object representing the name of the host.

Example:

```
using System;
using System.Net;

class DNSIPInfo
{
   public static void Main()
   {
           string hostName = Dns.GetHostName();
           Console.WriteLine("Local hostname: {0}", hostName);
           IPHostEntry myself = Dns.GetHostByName(hostName);
           foreach (IPAddress address in myself.AddressList)
           {
                Console.WriteLine("IP Address: {0}",
address.ToString());
           }
   }
}
```

```
Local hostname: ics-bmghandi
IP Address: 196.1.65.143
Press any key to continue . . .
```

GetHostByAddress work in the same manner, except that it takes IP address as argument.

The ***Resolve*** can take both hostname and IP address as argument.